20.2 Telegram 通道（grammY）

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~600k tokens，输出 ~55k tokens（本章合计）

Telegram 是 OpenClaw 支持最完善的通道之一——它的 Bot API 设计良好、文档完善，且支持丰富的消息格式。OpenClaw 使用 **grammY** 框架对接 Telegram Bot API。

20.2.1 grammY 框架与 Bot API 对接

> **衍生解释：grammY**
>
> grammY 是一个现代化的 Telegram Bot 框架，基于 TypeScript 编写，提供类型安全的 API 封装、中间件系统和插件生态。与 Telegraf 等老牌框架相比，grammY 的类型推导更完善，生态更活跃。

OpenClaw 的 Telegram 模块位于 `src/telegram/` 目录，核心文件是 `bot.ts` 。Bot 的创建过程涉及多个组件的协调：

```
// src/telegram/bot.ts（简化）
import { Bot, webhookCallback } from "grammy";
import { sequentialize } from "@grammyjs/runner";
import { apiThrottler } from "@grammyjs/transformer-throttler";

export function createTelegramBot(options: TelegramBotOptions) {
  const bot = new Bot(options.token);

  // 安装 API 限流器
  bot.api.config.use(apiThrottler());

  // 安装顺序化中间件——确保同一聊天的更新串行处理
  bot.use(sequentialize(getTelegramSequentialKey));

  // 注册原生命令
  registerTelegramNativeCommands(bot, ...);

  // 注册消息处理器
  registerTelegramHandlers(bot, ...);

  return bot;
}
```

顺序化处理

`sequentialize` 中间件确保来自**同一聊天**的更新串行处理，避免竞态条件。顺序键的生成逻辑考虑了多种上下文：

```
// src/telegram/bot.ts（简化）
export function getTelegramSequentialKey(ctx): string {
  // 优先使用 chat.id
  const chatId = ctx.chat?.id
    ?? ctx.update?.message?.chat?.id
    ?? ctx.update?.callback_query?.message?.chat?.id;

  if (chatId) return String(chatId);
  return ctx.me?.id ? `bot-${ctx.me.id}` : "global";
}
```

API 限流

`apiThrottler()` 插件自动对 Telegram API 调用进行限流，避免触发 Telegram 的速率限制（429 Too Many Requests）。对于多群组、多用户的高并发场景，这个插件是必不可少的。

20.2.2 Telegram Bot 配置与 Webhook 模式

Telegram Bot 支持两种接收更新的模式：

模式	机制	适用场景
长轮询（Long Polling）	Bot 主动向 Telegram 服务器拉取更新	开发环境、NAT 后的服务器
Webhook	Telegram 服务器向 Bot 推送更新	生产环境、公网可达的服务器

OpenClaw 两者都支持。Webhook 模式通过 `webhookCallback` 注册：

```
// src/telegram/webhook.ts（概念）
import { webhookCallback } from "grammy";

// 注册 Webhook 端点
const handleUpdate = webhookCallback(bot, "express");
app.post(`/webhook/${token}`, handleUpdate);

// 设置 Webhook URL
await bot.api.setWebhook(webhookUrl);
```

Webhook 模式的优势在于**低延迟**——Telegram 服务器在收到消息后立即推送，而长轮询需要等待下一次拉取周期。

配置还包括代理支持（`src/telegram/proxy.ts`）和网络错误处理（`src/telegram/network-errors.ts`），使 Bot 能在网络受限的环境中稳定运行。

20.2.3 草稿流特有实现

Telegram 的草稿流（Draft Stream）已在第 10.4 节详细讨论。此处补充其在 Bot 消息分发层的集成方式。

在 `bot-message-dispatch.ts` 中，草稿流的启用条件是：

```
const canStreamDraft =
  streamMode !== "off" &&
  isPrivateChat &&                    // 仅私聊
  typeof draftThreadId === "number" && // 有线程 ID
  (await resolveBotTopicsEnabled(primaryCtx)); // Topics 功能已启用
```

流模式（`streamMode`）通过 `resolveTelegramStreamMode` 从配置中解析，支持 `"off"`、`"partial"` 和 `"block"` 三种值。

Telegram 还有专门的草稿分块配置（`src/telegram/draft-chunking.ts`），用于 block 模式下的 BlockChunker 参数解析。

20.2.4 自定义命令

Telegram Bot 支持原生的斜杠命令（`/start`、`/help` 等）。OpenClaw 允许用户在配置中定义自定义命令：

```
// src/config/telegram-custom-commands.ts
export const TELEGRAM_COMMAND_NAME_PATTERN = /^[a-z0-9_]{1,32}$/;

export function resolveTelegramCustomCommands(params) {
  const entries = params.commands ?? [];
  const resolved = [];
  const issues = [];

  for (const entry of entries) {
    const normalized = normalizeTelegramCommandName(entry.command);

    // 校验：只允许小写字母、数字、下划线，最长 32 字符
    if (!TELEGRAM_COMMAND_NAME_PATTERN.test(normalized)) {
      issues.push({ message: `"/${normalized}" is invalid` });
      continue;
    }

    // 检查：不能与原生命令冲突
    if (reserved.has(normalized)) {
      issues.push({ message: `"/${normalized}" conflicts with native command` });
      continue;
    }

    // 检查：不能重复
    if (seen.has(normalized)) {
      issues.push({ message: `"/${normalized}" is duplicated` });
      continue;
    }

    resolved.push({ command: normalized, description });
  }

  return { commands: resolved, issues };
}
```

自定义命令的处理经过四层校验：格式合法性 → 保留字冲突 → 重复检测 → 描述完整性。这确保了提交给 Telegram Bot API 的命令列表始终是有效的。

原生命令注册在 `bot-native-commands.ts` 中，内置命令包括 `/start` （开始对话）、 `/help` （帮助）、 `/model` （切换模型）等。命令的授权检查可以配置为仅允许 bot 拥有者执行。

本节小结

1. **grammY 框架**提供类型安全的 Telegram Bot API 封装，OpenClaw 使用其顺序化中间件和 API 限流器。
2. 支持**长轮询和 Webhook** 两种模式，生产环境推荐 Webhook 以获得低延迟。
3. **草稿流**仅在私聊中启用，支持 partial 和 block 两种流模式。
4. **自定义命令**经过格式、冲突、重复、描述四层校验后注册到 Telegram Bot API。